DataJam - Team ProductDetails.

Bienvenidos a The Huddle: Data Integration Challenge Acaban de ser contratados como el nuevo equipo de Ingeniería de Datos de un e-commerce internacional en plena expansión. Sin embargo, su primer día arranca con una crisis: el equipo anterior abandonó el proyecto a la mitad.

Lograron exportar el historial de ventas transaccionales y el catálogo base en formato `.csv`, pero no existe una base de datos relacional operativa y, lo que es peor, **una dimensión crítica de la información se perdió por completo**.

El objetivo de su equipo para las próximas horas es reconstruir la infraestructura, modelar los datos y desarrollar una integración con servicios externos para rescatar la información faltante y dejar el sistema 100% operativo para responder a las preguntas de negocio.

🎯 La Misión (Tiempo estimado: 5 a 6 horas)

Para tener éxito en este desafío, deberán dividirse las tareas y completar las siguientes fases:

1. **Infraestructura y Setup:** Deberán instalar, configurar y levantar el motor de base de datos relacional que le fue asignado a su equipo.
2. **Modelado Relacional (DDL):** Analizar los archivos `.csv` del núcleo entregados (`users`, `products`, `orders`, `order_items`), diseñar el modelo relacional y escribir los scripts SQL (`CREATE TABLE`) asegurando las *Primary Keys* (PK), *Foreign Keys* (FK) y los tipos de datos correctos.
3. Creación del Diagrama UML y archivo de Schema.
4. **Migración Inicial (ETL):** Cargar los datos transaccionales base en su nueva base de datos. Si el modelado del paso anterior es correcto, la base de datos debería aceptar las relaciones sin arrojar errores.
5. **Integración de Datos Faltantes (Consumo de API):** En su paquete de datos encontrarán una tabla vital que está vacía o severamente incompleta. Deberán investigar la documentación de la API pública asignada a su grupo y escribir un script (en Python, Node.js, etc.) que consuma esos datos externos y ejecute los `INSERT` o `UPDATE` necesarios en su base de datos para completar el sistema.

🌟 BONUS TRACK: El Desafío Multimoneda (Opcional pero jugoso 💸)

Actualmente todas las ventas en la tabla `orders` están clavadas en dólares (USD). Para el equipo de finanzas, analizar esto a nivel local es un dolor de cabeza.

La Tarea: En su paquete de archivos notarán que hay una plantilla vacía llamada `currency_rates_template.csv`. Su misión extra es investigar y consumir una API pública de tipos de cambio (como *Frankfurter*, *ExchangeRate-API*, o la que prefieran encontrar) para obtener las tasas de conversión actuales.

El Entregable Extra:

1. Cargar las tasas de conversión desde la API hacia la tabla `currency_rates` en su base de datos.
2. Escribir una vista (`CREATE VIEW`) en SQL que cruce las órdenes de compra con esta nueva tabla y devuelva: el ID del pedido, el nombre del usuario, el monto original en USD, y el monto final convertido a otra moneda (por ejemplo, a Guaraníes - PYG, o directamente a la moneda local del usuario si se animan a cruzarlo con su tabla de países).

⚠️ Reglas de Trabajo y Restricciones

- **Prohibida la edición manual:** No está permitido abrir los archivos `.csv` originales para modificarlos a mano (por ejemplo, usando Excel). Toda la limpieza, carga y enriquecimiento debe hacerse estrictamente vía código o SQL.
- **Paralelización:** Son un equipo. Mientras una parte del grupo levanta el motor relacional y diseña las tablas, la otra parte puede (y debe) ir leyendo la documentación de la API y probando el código de extracción.
- **El objetivo final no es el código, es el dato:** Al faltar aproximadamente 90 minutos para el final de la jornada, se les entregará un set de **Consultas SQL de Negocio**. Su sistema será evaluado en base a su capacidad para ejecutar esas consultas con éxito y devolver resultados coherentes. Si la integración de la API falla, el sistema no podrá responder.

Instrucciones clave:

- **Motor Asignado:** MySQL / MariaDB. Deberán desplegar el servidor, asegurar los puertos y armar el esquema relacional asegurando la integridad referencial.
- **La Falla del Sistema:** Contamos con el catálogo base de ventas (`products`), pero la tabla operativa `product_details` colapsó. Perdimos el peso, el stock actual y el rating de los compradores.
- **El Objetivo:** Su script debe tomar el catálogo de productos de nuestra base, consultar la API de **DummyJSON Products** (https://dummyjson.com/products), y extraer las métricas exactas para reconstruir la tabla de detalles.
- **Regla de Oro:** No adivinen datos ni modifiquen los CSV a mano. Todo debe fluir desde la API hacia la base relacional mediante código. El sistema debe quedar 100% operativo para la fase de consultas.